

ПРАКТИЧЕСКОЕ ЗАНЯТИЕ 4.1.2 (ПЗ-12)

Представление данных и его влияние на ресурсы

Методическая разработка для студентов

Дисциплина: Системы обработки больших данных

Направление подготовки: 09.04.04 Программная инженерия

Уровень: магистратура

Форма работы: лабораторно-практическое занятие в Jupyter Notebook

Логическое продолжение ПЗ 4.1.1: если ранее диагностировались узкие места при чтении и загрузке данных, то теперь исследуется влияние способа представления признаков на память, размерность и время обучения.

1. Место работы в структуре темы

Практическое занятие 4.1.2 продолжает линию темы 4, посвящённой работе с большими данными на одном компьютере. В предыдущем занятии студенты сравнивали полную загрузку, пофайловую обработку и чтение по чанкам, то есть анализировали поведение системы на этапе доступа к данным. В текущей работе акцент переносится на следующий уровень: данные уже прочитаны, но необходимо понять, как их представление в памяти и в признаковом пространстве влияет на ресурсы вычислительной системы.

С методической точки зрения это принципиально важно. На практике многие затруднения возникают не только потому, что файл «большой», но и потому, что после one-hot-кодирования, текстовой векторизации или расширения признакового пространства исходная таблица превращается в гораздо более крупную матрицу признаков. Поэтому студент должен научиться анализировать не только размер исходных данных, но и размер, плотность и структуру представления, которое будет непосредственно использоваться алгоритмом обучения.

Цель занятия – показать, что выбор dense- или sparse-формата, а также контроль размерности признакового пространства являются инженерными решениями, напрямую влияющими на потребление памяти и время вычислений.

2. Цель, задачи и планируемые результаты

Цель работы состоит в том, чтобы научить студента сравнивать различные способы представления данных и делать инженерный вывод о том, какой формат признаков является предпочтительным для конкретного типа задачи.

В ходе работы студент должен решить следующие задачи:

- сопоставить dense- и sparse-представления на числовых, категориальных и текстовых данных;
- оценить, как one-hot-кодирование и текстовая векторизация увеличивают число признаков;
- измерить объём памяти, занимаемый плотными и разрежёнными матрицами;
- сравнить время предобработки и обучения модели для разных представлений;
- исследовать влияние роста размерности на время векторизации и обучения;
- сформулировать вывод о предпочтительном представлении признаков в каждом из рассмотренных сценариев.

Планируемые результаты освоения практической работы:

- объяснять различие между плотным и разрежённым представлением данных;
- распознавать ситуации, в которых one-hot- и текстовые признаки формируют разрежённые матрицы;
- оценивать память и плотность признаковой матрицы;
- связывать рост размерности с затратами времени на предобработку и обучение;
- делать обоснованный инженерный вывод о выборе представления данных.

3. Программные средства и состав файлов

Практическая работа выполняется в Jupyter Notebook. Все пути в ноутбук заданы относительно текущей папки, поэтому ноутбук и все данные должны располагаться в одном каталоге.

Файл	Назначение	Краткая характеристика
PZ_4_1_2_Представление_данных_и_ресурсы.ipynb	Основной рабочий ноутбук	Содержит код, пояснения, таблицы результатов и шаблоны выводов
numeric_profiles.csv	Числовой набор	Используется как dense-базовая матрица признаков
events_categorical.csv	Категориальный набор	Используется для one-hot-кодирования и сравнения dense/sparse
messages_text.csv	Текстовый набор	Используется для TF-IDF-представления и анализа размерности
variant_specs_pz_4_1_2.json	Таблица вариантов	Содержит параметры 20 индивидуальных заданий

4. Краткие теоретические сведения

4.1. Dense- и sparse-представления

Dense-представление предполагает, что в памяти хранятся все значения матрицы, включая нули. Такой формат удобен, когда большинство ячеек содержат полезную информацию. В задачах с числовыми признаками dense-представление обычно естественно и удобно: матрица компактна, понятна и легко обрабатывается стандартными средствами NumPy и pandas.

Sparse-представление используется тогда, когда в матрице очень много нулей и хранить их в явном виде невыгодно. В этом случае сохраняются только ненулевые элементы и их позиции. На практике это особенно важно для текстовых данных и one-hot-кодирования категориальных признаков, где число столбцов быстро растёт, а каждая отдельная строка содержит единицы лишь в малом числе позиций.

4.2. One-hot-кодирование

При one-hot-кодировании каждая категория заменяется отдельным столбцом. Если в признаке 100 разных значений, то после кодирования появляется 100 бинарных столбцов. Когда одновременно кодируется несколько категориальных признаков высокой кардинальности, число столбцов быстро возрастает. Именно поэтому one-hot-кодирование является типичным источником роста размерности.

4.3. Текстовая векторизация

Текстовая векторизация, например TF-IDF, преобразует набор документов в матрицу «документ–термин». Каждый столбец соответствует слову или n-грамме. Большинство документов содержит лишь малую часть общего словаря, поэтому матрица признаков обычно оказывается разрежённой. Переход к dense-представлению в такой задаче возможен только на небольших наборах и чаще всего служит учебным сравнением, а не промышленным решением.

4.4. Размерность признакового пространства

Рост числа признаков влияет не только на память, но и на время предобработки, обучения и прогнозирования. Даже если модель остаётся линейной, увеличение числа столбцов означает рост числа операций и более дорогую работу с матрицей. Поэтому контроль размерности – это не только вопрос качества модели, но и вопрос вычислительной эффективности.

5. Алгоритм выполнения работы

1. Откройте ноутбук и установите номер варианта в переменной `VARIANT`.
2. Изучите таблицу параметров варианта и обратите внимание на индивидуальный аналитический акцент.
3. Выполните эксперимент 1 и зафиксируйте память и время для числового dense-представления.
4. Выполните эксперимент 2 и сравните dense one-hot и sparse one-hot на категориальных данных.
5. Выполните эксперимент 3 и сравните sparse TF-IDF и dense TF-IDF на текстах.
6. Выполните эксперимент 4 по сетке значений `max\_features` и проследите влияние размерности на ресурсы.
7. Сведите результаты в общую сравнительную таблицу.
8. Сформулируйте письменные выводы по каждому эксперименту и общий инженерный вывод.

6. Подробное содержание экспериментов

6.1. Эксперимент 1. Числовые признаки как dense-базис

Первый эксперимент служит контрольной точкой. Студент работает с числовым набором и убеждается, что dense-представление здесь естественно и экономически оправдано. В отчёте необходимо показать объём памяти DataFrame и NumPy-массива, а также время обучения линейной модели.

При анализе результатов студент должен обратить внимание на то, что высокая плотность матрицы делает sparse-представление бессмысленным или малоэффективным. Следовательно, сам факт существования sparse-форматов ещё не означает, что их следует использовать всегда.

6.2. Эксперимент 2. Категориальные признаки и one-hot-кодирование

Во втором эксперименте формируется бинарная матрица признаков из категориальных столбцов. Здесь необходимо сопоставить две версии одной и той же информации: dense one-hot и sparse one-hot. Особое внимание следует уделить числу столбцов, плотности матрицы и объёму памяти.

Студент должен показать, что логика кодирования не меняется, однако способ хранения признаков меняет ресурсоёмкость задачи. Если dense-версия хранит весь массив нулей и единиц, то sparse-версия использует только ненулевые элементы и их позиции.

6.3. Эксперимент 3. Текстовые признаки

Третий эксперимент переносит сравнение в область текстовых данных. На этом этапе студент получает естественно разрежённую TF-IDF-матрицу. Dense-версия строится только для учебного сравнения и позволяет численно увидеть, насколько дороже становится хранение текста в полном виде.

Именно этот эксперимент помогает лучше всего почувствовать, почему работа с текстом практически всегда требует компактного представления признаков.

6.4. Эксперимент 4. Эффект размерности

В четвёртом эксперименте изменяется параметр `'max_features'`. Это позволяет наблюдать, как растёт число признаков и как вслед за ним изменяются объём памяти, время векторизации и время обучения. В отчёте следует дать не только количественное описание изменений, но и интерпретацию: почему рост словаря признаков увеличивает ресурсоёмкость системы.

7. Индивидуальные задания (20 вариантов)

Индивидуальные варианты отличаются объёмом данных, параметрами текстовой векторизации и аналитическим акцентом. Все варианты выполняются на одном и том же наборе файлов, но в ноутбуке автоматически выбираются разные подмножества строк и разные параметры обработки.

Вар.	Числовые строки	Категориальные строки	Текстовые строки	max_features	ngram	min_df
1	2500	3000	900	500	(1, 1)	2
2	2700	3200	1000	600	(1, 1)	2
3	2900	3400	1100	700	(1, 1)	2
4	3100	3600	1200	800	(1, 1)	3
5	3300	3800	1300	900	(1, 1)	3
6	3500	4000	1400	1000	(1, 2)	2
7	3700	4200	1500	1100	(1, 2)	2
8	3900	4400	1600	1200	(1, 2)	3
9	4100	4600	1700	1300	(1, 2)	3
10	4300	4800	1800	1400	(1, 2)	4
11	4500	5000	1900	1500	(1, 1)	2

12	4700	5200	2000	1600	(1, 1)	3
13	4900	5400	2100	1700	(1, 2)	2
14	5100	5600	2200	1800	(1, 2)	3
15	5300	5800	2300	1900	(1, 1)	4
16	5500	6000	2400	1000	(1, 2)	2
17	5700	6200	2500	1200	(1, 1)	3
18	5900	6400	2600	1400	(1, 2)	4
19	6100	6600	2700	1600	(1, 1)	2
20	6300	6800	2800	1800	(1, 2)	3

Индивидуальные аналитические акценты вариантов:

1. **Вариант 1.** Сделайте основной акцент на сравнении плотной и разрежённой one-hot матрицы.
2. **Вариант 2.** Обратите внимание на влияние числа категорий на размер dense-представления.
3. **Вариант 3.** Сравните, как изменяются память и время при росте числа текстовых признаков.
4. **Вариант 4.** Особое внимание уделите этапу векторизации текста и последующему обучению.
5. **Вариант 5.** Сделайте вывод о том, где выгоднее использовать компактное кодирование признаков.
6. **Вариант 6.** Сравните unigram и bigram-представления по плотности и затратам ресурсов.
7. **Вариант 7.** Отдельно прокомментируйте, почему sparse-матрица удобнее для текстовых данных.
8. **Вариант 8.** Покажите, как high-cardinality категориальные признаки раздувают dense-матрицу.
9. **Вариант 9.** Сделайте акцент на различии между числовыми и текстовыми признаками.
10. **Вариант 10.** Отдельно прокомментируйте связь между размерностью и временем обучения модели.
11. **Вариант 11.** Поясните, насколько оправдан перевод sparse-матрицы в dense-вид.
12. **Вариант 12.** Проведите анализ того, где появляется наибольший прирост памяти: при кодировании или при обучении.
13. **Вариант 13.** Сфокусируйтесь на сравнении полных и компактных представлений категориальных признаков.
14. **Вариант 14.** Сравните, как меняются результаты при росте max_features в текстовом векторизаторе.
15. **Вариант 15.** Особое внимание уделите плотности матрицы признаков и её интерпретации.
16. **Вариант 16.** Поясните, как изменение ngram_range влияет на число признаков и стоимость обучения.
17. **Вариант 17.** Покажите различие между хранением данных и хранением матрицы признаков.
18. **Вариант 18.** Прокомментируйте, почему sparse-представление важно именно в задачах высокой размерности.
19. **Вариант 19.** Сделайте вывод о компромиссе между удобством dense-матрицы и эффективностью sparse-формата.

20. Вариант 20. Дайте сводный вывод о том, какие представления признаков предпочтительны для разных типов данных.

8. Требования к отчёту

- тема, цель работы и номер варианта;
- краткое объяснение различия между dense- и sparse-представлениями;
- таблицы результатов по всем четырём экспериментам;
- сравнительный анализ памяти, плотности и времени выполнения;
- ответ на индивидуальный аналитический акцент варианта;
- общий инженерный вывод о предпочтительном представлении признаков для числовых, категориальных и текстовых данных.

9. Контрольные вопросы

21. Почему размер исходного файла не совпадает с размером построенной матрицы признаков?
22. Почему one-hot-кодирование часто приводит к разрежённой матрице?
23. Почему текстовые данные почти всегда удобнее хранить в sparse-формате?
24. Как рост числа признаков влияет на время предобработки и обучения?
25. В каких случаях dense-представление остаётся предпочтительным?
26. Как результаты ПЗ 4.1.2 (ПЗ-12) дополняют выводы ПЗ 4.1.1(ПЗ-11)?

10. Итоговый вывод, которого должен добиться студент

В результате выполнения работы студент должен прийти к выводу, что выбор представления данных является не вспомогательной, а центральной инженерной задачей. Dense-представление удобно для компактных числовых данных с высокой заполненностью, тогда как one-hot- и текстовые признаки формируют разрежённые матрицы, для которых sparse-хранение даёт существенную экономию памяти. Рост размерности повышает стоимость предобработки и обучения, поэтому контроль числа признаков становится важной частью проектирования аналитической системы.